Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 25.М71-мм

Разреженная линейная алгебра и Interaction Nets: рефакторинг системы измерения производительности

ПОНОМАРЕВ Николай Алексеевич

Отчёт по производственной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Григорьев С. В.

Санкт-Петербург
2026

Оглавление

Введение **3**

1. Постановка задачи **4**

2. Обзор существующих решений **5**

3. Проектирование и реализация **6**

3.1. Единый интерфейс запуска 6

3.2. Настройки и общие модули 7

3.3. Работа с матрицами 7

3.4. Подключение инструментов 8

3.5. Расширение инфраструктуры 9

4. Заключение **10**

Список литературы **11**

Введение

Многие прикладные алгоритмы — например, в областях искусственного интеллекта и анализа графов — естественным образом выражаются в терминах линейной алгебры. Такой подход позволяет использовать готовые библиотеки, например, BLAS [6] для плотных матриц и GraphBLAS [3] для разреженных, использующие в том числе большие возможности линейной алгебры по распараллеливанию вычислений.

Если в случае плотных матриц основными ограничениями являются объём памяти, необходимый для хранения, и скорость передачи данных, то в случае разреженных основной проблемой являются нерегулярный параллелизм, выражающийся в нелокальных обращениях к памяти и непредсказуемом количестве независимых подзадач, в силу принципов проектирования современных высокопроизводительных CPU и GPU.

Interaction Nets — модель вычислений, которая была описана Yves Lafont в 1989 году [5]. В этой модели программа представляется в виде графа, а одним из важных свойств является естественная поддержка нерегулярного параллелизма. Из-за чего Interaction Nets может быть перспективной основой специализированных ускорителей.

Существующие реализации исполнителей Interaction Nets демонстрируют скорость исполнения лишь на базовых конструкциях (сортировка, обход списков). В рамках проекта Lamagraph[1] изучаются возможности по реализации алгоритмов разреженной линейной алгебры на Interaction Nets.

Для оценки применимости Interaction Nets был разработан небольшой инструмент для сравнения производительности различных реализаций алгоритмов линейной алгебры: изначально разрозненный набор скриптов, с дублирующимся кодом и отсутствием документации. В данной работе предлагается произвести рефакторинг данного инструмента, путём унификации общего кода, выделения единообразного интерфейса, а также создания документации по расширению другими реализациями.

[1]Репозитории проекта доступны по ссылке: https://github.com/Lamagraph/ (дата обращения: 18 мая 2026 г.)

1. Постановка задачи

Целью настоящей работы является реорганизация инфраструктуры для измерения производительности графовых алгоритмов на основе Interaction Nets.

Для её выполнения были поставлены следующие задачи:

1. Объединить разрозненные скрипты в одну программу с командами получения матриц, их подготовки и запуска измерений.
2. Задать состав измерений одним файлом настроек и автоматизировать подготовку матриц.
3. Перенести в новую инфраструктуру модули всех используемых инструментов и описать правила подключения новых инструментов.

2. Обзор существующих решений

Настоящая работа посвящена реорганизации инфраструктуры для измерения производительности графовых алгоритмов на основе Interaction Nets [5]. Под инфраструктурой здесь понимается совокупность скриптов, настроек и соглашений, автоматизирующих подготовку тестовых матриц, запуск измерений и сбор результатов для набора инструментов.

В качестве объектов сравнения рассматриваются реализации на основе Interaction Nets: QTreeInpla[2], QTreeVine[3], а также однопоточная реализация на F#[4], являющаяся их прототипом. Кроме того, используются эталонные инструменты: NetworkX[5] и LAGraph[6], построенный поверх SuiteSparse:GraphBLAS [3].

Инфраструктура поддерживает три алгоритма: обход в ширину (BFS) [1], поиск кратчайших путей из источника (SSSP) [4] и подсчёт треугольников (Triangle Counting) [2].

Ранее инфраструктура состояла из набора отдельных скриптов в каталоге `scripts/`. Такая организация имела ряд недостатков: логика загрузки и подготовки матриц дублировалась между скриптами, набор матриц и инструментов не был описан в одном месте, добавление нового инструмента требовало правок в нескольких файлах, а результаты измерений не имели единого формата и могли оказаться несовместимы с кодом построения графиков.

Реорганизация была направлена на устранение этих недостатков: единый интерфейс запуска, общие модули, формализованные правила подключения инструментов, автоматизация работы с матрицами и единое соглашение о хранении результатов.

---

[2]Реализация доступна по ссылке: https://github.com/Lamagraph/QTreeInpla (дата обращения: 2 июня 2026 г.)

[3]Реализация доступна по ссылке: https://github.com/Lamagraph/QTreeVine (дата обращения: 2 июня 2026 г.)

[4]Реализация доступна по ссылке: https://github.com/Lamagraph/QTreeFSharp (дата обращения: 2 июня 2026 г.)

[5]Реализация доступна по ссылке: https://networkx.org/ (дата обращения: 2 июня 2026 г.)

[6]Реализация доступна по ссылке: https://github.com/GraphBLAS/LAGraph (дата обращения: 2 июня 2026 г.)

3. Проектирование и реализация

Инфраструктура реализована на Python 3.13, интерфейс командной строки построен на библиотеке Typer, зависимости управляются через `uv`. Состав измерений хранится в YAML и загружается в типизированные структуры, результаты записываются в JSON. Инструменты и их зависимости подключены с помощью Git Submodules. Устройство инфраструктуры показано на рисунке 1, порядок работы с ней — на рисунке 2.

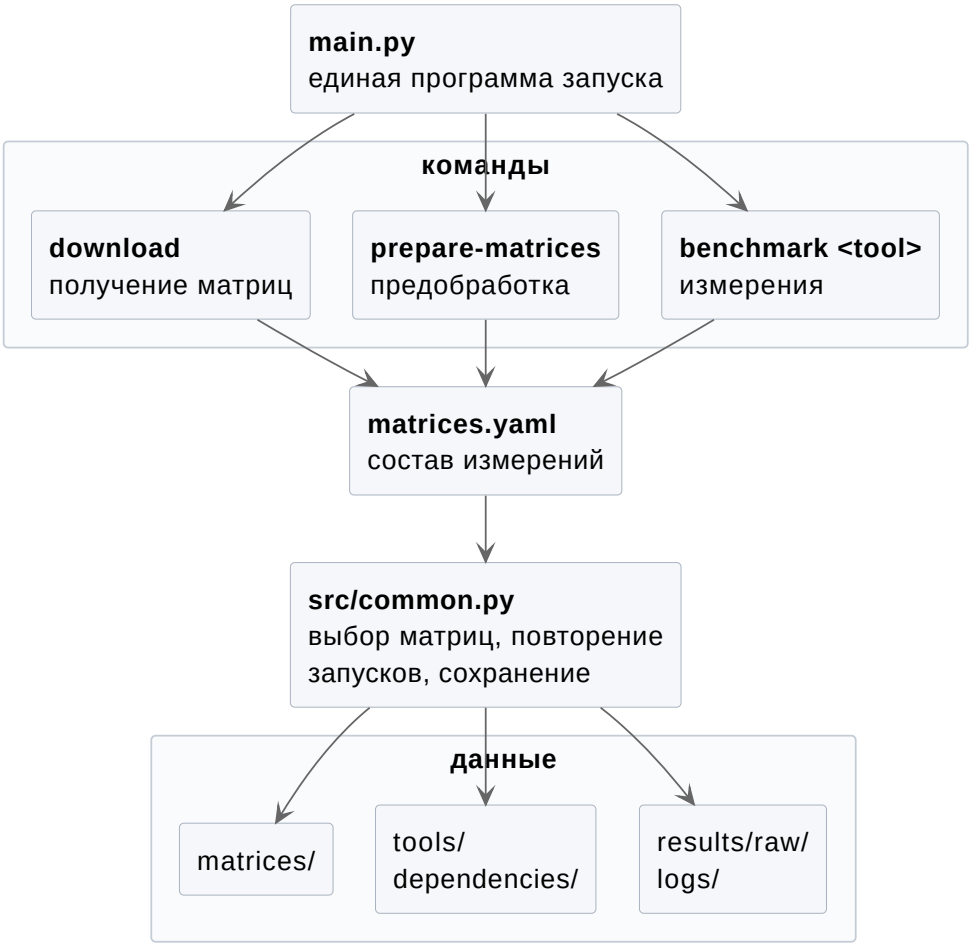


Рис. 1: Устройство инфраструктуры для измерения производительности

3.1. Единый интерфейс запуска

Все этапы работы объединены в одной программе с тремя командами: `download` получает тестовые матрицы, `prepare-matrices` выполняет их

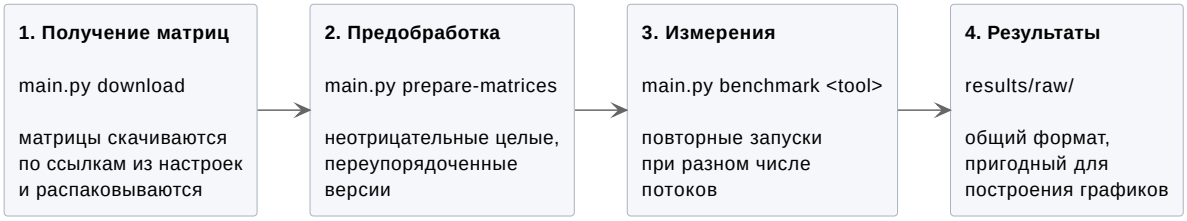


Рис. 2: Порядок работы с инфраструктурой

предобработку, а `benchmark <tool>` запускает измерения для выбранного инструмента. Каждая команда ведёт общий журнал работы, который выводится на экран и одновременно сохраняется в файл.

3.2. Настройки и общие модули

Набор матриц, алгоритмов и инструментов задаётся в одном файле `matrices.yaml`: для каждой матрицы указываются алгоритм, список инструментов, необходимость переупорядочения и ссылка для скачивания. Формат выбран по составу данных: записи о матрицах содержат вложенные списки инструментов и логические пометки, поэтому плоский CSV, использовавшийся в первой версии, для них неудобен, а JSON не допускает комментариев. YAML поддерживает и то, и другое и остаётся читаемым при ручной правке. Состав измерений меняется правкой одного файла.

Общая для всех инструментов логика вынесена в отдельный модуль `src/common.py`: выбор подходящих матриц, повторение запусков, перебор числа потоков и сохранение результатов. Результаты каждого инструмента записываются в отдельный файл, имя которого содержит время запуска и название инструмента; такое именование использует код построения графиков, поэтому все инструменты сохраняют данные одинаковым образом.

3.3. Работа с матрицами

Команда `download` читает список матриц из файла настроек, отбрасывает повторы, скачивает архивы по указанным ссылкам и распаковывает

их в общий каталог. Уже загруженные матрицы повторно не скачиваются, поэтому прерванную загрузку можно безопасно продолжить.

Команда `prepare-matrices` выполняет две операции. Первая — приведение матриц к общему виду: значения усекаются до целых, а при наличии отрицательных элементов все значения сдвигаются на $|\min|$, чтобы матрица стала неотрицательной. Вторая — переупорядочение по алгоритму обратного Катхилла—Макки (RCM), которое выполняется для матриц, помеченных в настройках: матрица симметризуется по образцу без диагонали, для неё вычисляется перестановка, которая применяется к строкам и столбцам; для прямоугольных матриц переставляются только строки. Результат сохраняется отдельным файлом.

3.4. Подключение инструментов

Модули для отдельных инструментов расположены в каталоге `src/benchmark/` и подключаются к единому интерфейсу как отдельные команды. На практике сложились два способа подключения.

Первый способ применяется, когда инструмент сам печатает время работы: программа запускается как внешний процесс с нужным числом потоков, а её вывод сохраняется и разбирается после завершения. Так подключены LAGraph и Inpla.

Второй способ применяется, когда инструмент располагает собственными средствами запуска измерений и формирует готовый отчёт: инфраструктура готовит для него матрицы, запускает его средства и переносит полученный отчёт в общий каталог результатов. Так подключены NetworkX и F#.

3.4.1. Особенности подключения Inpla

При подключении Inpla основная сложность связана с управлением памятью. Интерпретатор хранит агенты и имена в кольцевых буферах, которые расширяются по мере заполнения, а параметр `-Xms` задаёт начальный размер этих буферов в ячейках. От начального размера зависит число обращений к системе за дополнительной памятью: при слишком

малом значении буферы расширяются многократно, и накладные расходы искажают измеряемое время, а при слишком большом — растёт потребление памяти и возрастает риск исчерпания доступной памяти на машине.

Каждый поток создаёт собственные буферы агентов и имён, поэтому общий объём памяти пропорционален числу потоков. Одна пара ячеек занимает 104 байта (`sizeof(Agent)` = 88, `sizeof(Name)` = 16), что позволяет перевести желаемый объём памяти в число ячеек. В работе используется формула

$$N = \frac{S \cdot 2^{30}}{104 \cdot 1.2},$$

где S — желаемый суммарный объём памяти в гибибайтах. Множитель 1.2 отводит часть этого объёма под прочие структуры интерпретатора, поэтому на ячейки приходится около $S/1.2$. При запуске с T потоками каждому потоку передаётся значение N/T, так что суммарный объём не превышает выбранного S. Значение по умолчанию $N = 1\,032\,444\,062$ соответствует $S = 120$ ГиБ и рассчитано на машину с 128 ГиБ памяти.

Дополнительно учитываются два ограничения. Интерпретатор отвергает запрос на буфер, превышающий 10^8 ячеек, поэтому при малом числе потоков ограничение сверху задаёт именно оно, а не объём памяти. Кроме того, на больших матрицах интерпретатор может исчерпать стек, поэтому перед запуском лимит стека снимается, а ненулевой код возврата не считается признаком неудачи: Inpla иногда завершается с ошибкой уже после вывода результата, и этот результат сохраняется.

3.5. Расширение инфраструктуры

Правила подключения нового инструмента описаны в документации для разработчиков. В общем случае нужно объявить инструмент в общем перечислении, добавить модуль запуска, подключить его к единому интерфейсу и указать инструмент в настройках для нужных матриц. Репозитории самих инструментов и их зависимости хранятся отдельно от кода инфраструктуры и обновляются независимо от него.

4. Заключение

В ходе работы была реорганизована инфраструктура для измерения производительности графовых алгоритмов на основе Interaction Nets: вместо набора разрозненных скриптов создана единая система с общим интерфейсом запуска.

В результате решения поставленных задач получены следующие результаты:

1. Все этапы работы объединены в одну программу с командами получения матриц, их подготовки и запуска измерений, а также с общим журналом работы; общая для всех инструментов логика вынесена в отдельный модуль.

2. Состав измерений полностью задаётся одним файлом настроек, а загрузка и предобработка матриц, включая построение переупорядоченных версий, выполняются автоматически.

3. К новой инфраструктуре подключены все используемые инструменты, включая особенности Inpla, и описаны единые правила подключения новых инструментов.

Реорганизованная инфраструктура сокращает дублирование кода, упрощает добавление новых инструментов и матриц, а также делает проведение и повторение измерений более удобным.

Pull Request с рефакторингом: https://github.com/Lamagraph/graph-benchmarks/pull/3, имя коммитера: `WoWaster`.

Список литературы

[1] Davis Timothy A. Algorithm 1000: SuiteSparse:GraphBLAS: Graph Algorithms in the Language of Sparse Linear Algebra // ACM Transactions on Mathematical Software. — 2019. — Vol. 45, no. 4. — P. 1–25.

[2] Fast linear algebra-based triangle counting with KokkosKernels / Michael M. Wolf, Mehmet Deveci, Jonathan W. Berry et al. // 2017 IEEE High Performance Extreme Computing Conference (HPEC). — IEEE, 2017. — P. 1–7.

[3] The GraphBLAS standard: parallel graph algorithms in the language of sparse linear algebra / Aydin Buluç, Timothy Mattson, Scott McMillan et al. // arXiv preprint arXiv:1501.05263. — 2015.

[4] Kepner Jeremy, Gilbert John. Graph Algorithms in the Language of Linear Algebra. — Philadelphia, PA, USA : Society for Industrial and Applied Mathematics, 2011. — ISBN: 978-0-89871-990-1.

[5] Lafont Yves. Interaction Nets // Proceedings of the 17th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages. — POPL '90. — New York, NY, USA : Association for Computing Machinery, 1989. — P. 95–108.

[6] An Updated Set of Basic Linear Algebra Subprograms (BLAS) / L. Susan Blackford, James Demmel, Jack Dongarra et al. // ACM Transactions on Mathematical Software. — 2002. — Vol. 28, no. 2. — P. 135–151.